

`npm ci` is a command used to install Node.js project dependencies in a **clean, fast, and reproducible way**, mainly for:

- CI/CD pipelines (Continuous Integration)
- Build servers
- Docker containers
- Production deployments

The `ci` stands for **Continuous Integration**.

Difference Between `npm install` and `npm ci`

Suppose your project contains:

```
package.json  
package-lock.json
```

`npm install`

```
npm install
```

Behavior:

1. Reads `package.json`
2. Reads `package-lock.json`
3. Installs dependencies
4. May update `package-lock.json`
5. May resolve newer compatible versions

Example:

```
{  
  "dependencies": {  
    "express": "^4.18.0"  
  }  
}
```

The `^` means:

Install 4.18.0 or newer compatible 4.x versions.

So different developers might end up with slightly different dependency trees over time.

`npm ci`

```
npm ci
```

Behavior:

1. Requires a `package-lock.json`
2. Deletes existing `node_modules`

3. Installs exactly what's in `package-lock.json`
4. Never modifies lock files
5. Fails if `package.json` and `package-lock.json` don't match

This guarantees everyone gets the exact same dependency versions.

Example

package.json

```
{
  "dependencies": {
    "express": "^4.18.0"
  }
}
```

package-lock.json

```
{
  "express": "4.18.2"
}
```

Running:

```
npm ci
```

installs:

```
express 4.18.2
```

exactly.

Not:

```
express 4.19.0
```

even if that version exists.

Why Delete `node_modules`?

Before installing, `npm ci` does:

```
rm -rf node_modules
```

(Conceptually.)

Then recreates everything from scratch.

This avoids problems caused by:

- leftover packages
- corrupted installs

- stale dependencies
-

Why CI/CD Uses It

Imagine a build server:

```
Developer
  |
  v
GitHub
  |
  v
CI Server
```

If the CI server uses:

```
npm install
```

dependencies might change unexpectedly.

Using:

```
npm ci
```

ensures:

```
Developer machine
=
Test machine
=
Production server
```

all use identical dependencies.

Docker Example

You'll often see:

```
COPY package*.json ./
RUN npm ci
COPY . .
```

Why?

Because:

- Faster
 - More reproducible
 - Better Docker layer caching
 - Predictable builds
-

When It Fails

Missing lock file

`npm ci`

Output:

npm ERR! The npm ci command can only install with an existing package-lock.json

Lock file mismatch

If:

`package.json`

says:

"express": "^5.0.0"

but

`package-lock.json`

contains:

"express": "4.18.2"

then:

`npm ci`

fails.

This is intentional.

Performance

Usually:

`npm ci`

is faster than:

`npm install`

because npm doesn't need to:

- resolve dependency versions
 - update lock files
 - reconcile existing `node_modules`
-

Common Workflow

Developer

```
npm install axios
```

This updates:

```
package.json  
package-lock.json
```

Commit both files:

```
git add package.json package-lock.json  
git commit
```

CI Server

```
npm ci  
npm test  
npm run build
```

Uses the exact versions committed by the developer.

Why You Might See It in Hyperledger Fabric Projects

Many Hyperledger Fabric applications use:

- Node.js SDKs
- Gateway applications
- Chaincode written in JavaScript/TypeScript

Their build scripts often contain:

```
npm ci
```

to ensure every peer, developer machine, and CI pipeline uses the exact same dependency versions.

Quick Summary

Command	Purpose
<code>npm install</code>	Install dependencies and update lock file if needed
<code>npm ci</code>	Clean install using exactly <code>package-lock.json</code>
Requires <code>package-lock.json</code>	No (<code>install</code>) / Yes (<code>ci</code>)
Deletes <code>node_modules</code> first	No (<code>install</code>) / Yes (<code>ci</code>)
Modifies lock file	Possible (<code>install</code>) / Never (<code>ci</code>)
Best for CI/CD & Docker	<code>npm ci</code>
A good rule of thumb:	

- **Developing locally:** use `npm install`
- **Automated builds, Docker, CI/CD:** use `npm ci`